Chapter 8

Graphs

1

Objectives

Discuss the following topics:

- Graphs
- Graph Representation
- Graph Traversals
- Shortest Paths
- Cycle Detection
- Spanning Trees – Phủ cây
- Connectivity
- Eulerian and Hamiltonian Graphs
- Graph Coloring

2

2

Introduction to Graphs

- A **graph** is a collection of vertices (or nodes) and the connections between them
- A **simple graph** $G = (V, E)$ consists of a nonempty set V of **vertices** and a possibly empty set E of **edges**, each edge being a set of two vertices from V
- A **directed graph**, or a **digraph**, $G = (V, E)$ consists of a nonempty set V of vertices and a set E of edges (also called **arcs**), where each edge is a pair of vertices from V

3

3

Graphs (continued)

- A **multigraph** is a graph in which two vertices can be joined by multiple edges
- A **pseudograph** is a multigraph with the condition $v_i \neq v_j$ removed, which allows for loops to occur
- If all vertices in a circuit are different, then it is called a **cycle**
- A graph is called a **weighted graph** if each edge has an assigned number

4

4

Graphs (continued)

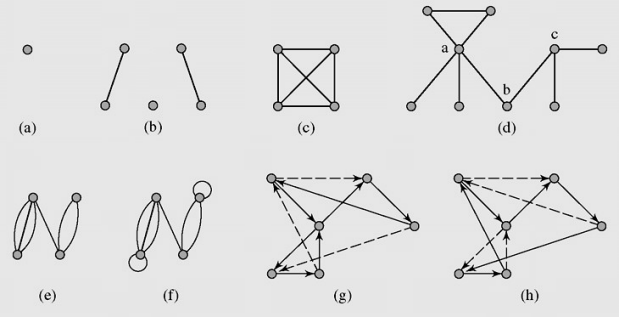


Figure 8-1 Examples of graphs: (a–d) simple graphs; (c) a complete graph K_4; (e) a multigraph; (f) a pseudograph; (g) a circuit in a digraph; (h) a cycle in the digraph

8.1- Graph Representation

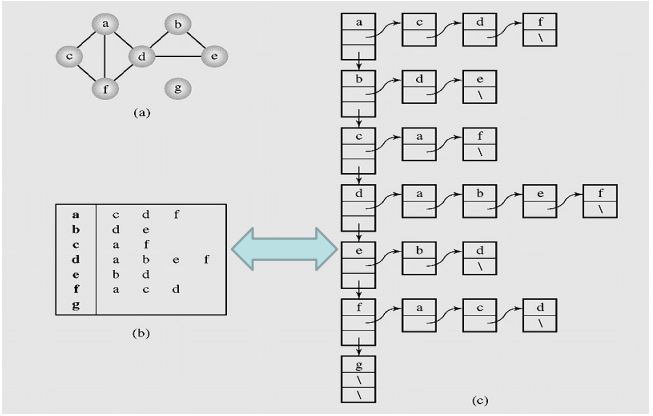
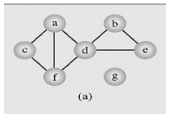


Figure 8-2 Graph representations (a) A graph represented as (b–c) an adjacency list



Graph Representation (continued)

	a	b	c	d	e	f	g
a	0	0	1	1	0	1	0
b	0	0	0	1	1	0	0
c	1	0	0	0	0	1	0
d	1	1	0	0	1	1	0
e	0	1	0	1	0	0	0
f	1	0	1	1	0	0	0
g	0	0	0	0	0	0	0

(d)

	ac	ad	af	bd	be	cf	de	df
a	1	1	1	0	0	0	0	0
b	0	0	0	1	1	0	0	0
c	1	0	0	0	0	1	0	0
d	0	1	0	1	0	0	1	1
e	0	0	0	0	1	0	1	0
f	0	0	1	0	0	1	0	1
g	0	0	0	0	0	0	0	0

(e)

Figure 8-2 Graph representations (d) an adjacency matrix, and (e) an incidence matrix (continued)

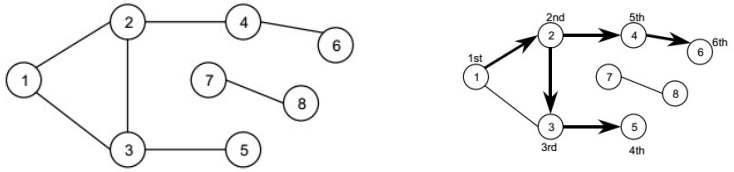
8.2- Graph Traversals

- In the **depth-first search algorithm**, each vertex *v* is visited and then each unvisited vertex adjacent to *v* is visited

```
void DFS (Vertex v)
    num(v) = i++;  // number represents the order in traversal, it marks that whether this vertex is visited nor not
    for all vertices u adjacent to v
        if (num(u) is 0)
                attach edge uv to edges;
                DFS(u);
```

```
void depthFirstSearch()
    for all vertices v  num(v) = 0;
    edges = null;
    i = 1;
    while there is a vertex v such that num(v) is 0 DFS(v);
    output edges;
```

Example



10

10

Graph Traversals

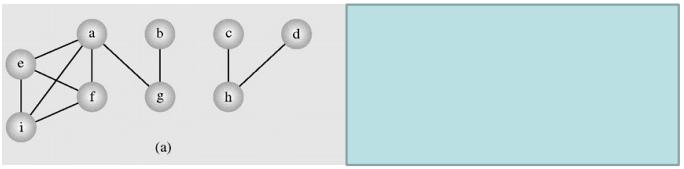


Figure 8-3 An example of application of the `depthFirstSearch()` algorithm to a graph

11

11

Graph Traversals (continued)

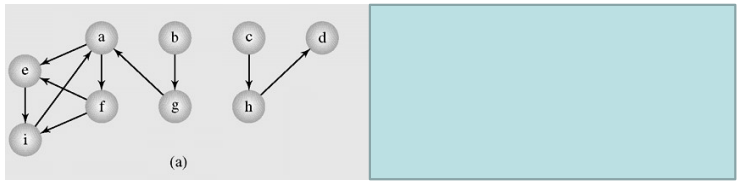
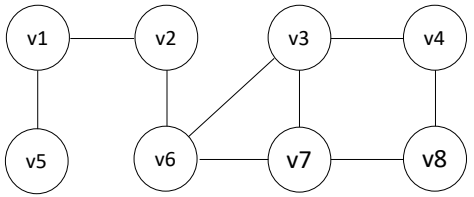


Figure 8-4 The `depthFirstSearch()` algorithm applied to a digraph

13

13

Example DFS

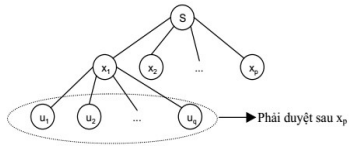


14

14

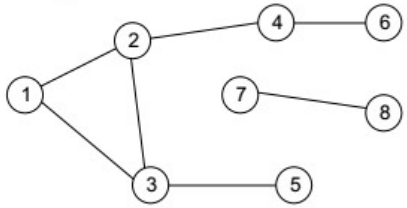
Graph Traversals (continued)

```
breadthFirstSearch()
    for all vertices v  num(v) = 0;
    edges = null;
    i = 1;
    while there is a vertex v such that num(v) is 0
            num(v) = i++;
            enqueue (v)
            while queue is not empty
                    v=dequeue();
                    for all vertices u adjecent to v
                            if num(u) is 0
                                    num(u) = i++;
                                    enqueue(u);
```



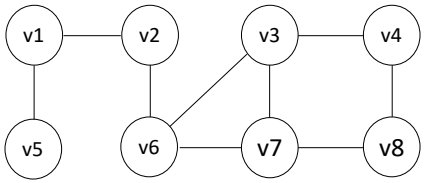
Example

Xét đồ thị dưới đây, Đỉnh xuất phát S = 1.

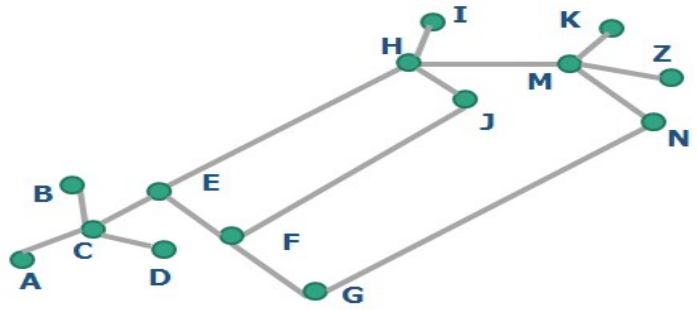


Hàng đợi	**Đỉnh u (lấy ra từ hàng đợi)**	Hàng đợi (sau khi lấy u ra)	Các đỉnh v kề u mà chưa lên lịch	Hàng đợi sau khi đẩy những đỉnh v vào
(1)	**1**	∅	2, 3	(2, 3)
(2, 3)	**2**	(3)	4	(3, 4)
(3, 4)	**3**	(4)	5	(4, 5)
(4, 5)	**4**	(5)	6	(5, 6)
(5, 6)	**5**	(6)	Không có	(6)
(6)	**6**	∅	Không có	∅

Example - BFS



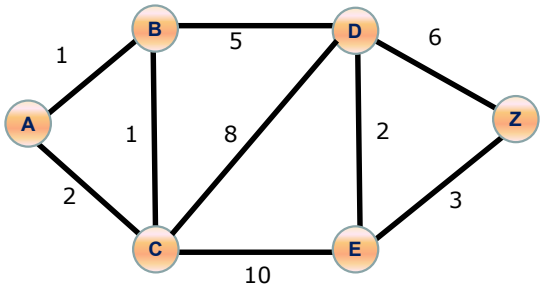
Exercise – DFS – BFS from A



Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9					

Weighted Graph

Tìm đường đi ngắn nhất từ A đến mỗi đỉnh khác của đồ thị G dưới đây



25

25

8.3- Shortest Paths

- The methods solving the shortest path problem are divided in two classes:
  - **Label-setting** methods
    - In each pass through the vertices still to be processed, one vertex is set to a value that remains unchanged to the end of the execution
  - **Label-correcting** methods
    - Allow for the changing of *any* label during application of the method
- **Negative cycle** is a cycle composed of edges with weights adding up to a negative number

26

26

Some Algorithm

- **Dijkstra**
- **Ford-Bellman**
- **Floyd-Warshall**

27

27

Shortest Paths (continued)

```
DijkstraAlgorithm(weighted simple digraph, vertex first)
    for all vertices v
        currDist(v) = ∞;
    currDist(first) = 0;
    toBeChecked = all vertices;
    while toBeChecked is not empty
        v = a vertex in toBeChecked with minimal currDist(v);
        remove v from toBeChecked;
        for all vertices u adjacent to v and in toBeChecked
            if currDist(u) > currDist(v)+ weight(edge(vu))
                currDist(u) = currDist(v)+ weight(edge(vu));
                predecessor(u) = v;
```

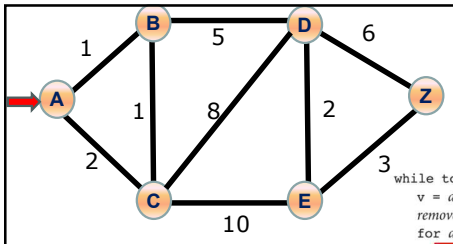
Use: Positive weight

Time complexity depends on the implementation:

- Can be $O(n^2 + m)$, $O(m \log n)$, or $O(m + n \log n)$

29

29



Tìm đường đi ngắn nhất từ A đến mỗi đỉnh khác của đồ thị G dưới đây

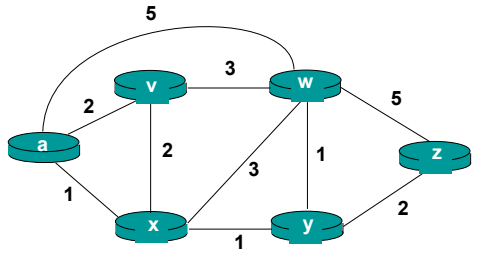
```
while toBeChecked is not empty
    v = a vertex in toBeChecked with minimal currDist(v);
    remove v from toBeChecked;
    for all vertices u adjacent to v and in toBeChecked
        if currDist(u) > currDist(v)+ weight(edge(vu))
            currDist(u) = currDist(v)+ weight(edge(vu));
            predecessor(u) = v;
```

Bước	Tập T	A	B	C	D	E	Z
0 (initial)	A,B,C,D,E,Z	(0,-)	(∞, -)	(∞, -)	(∞, -)	(∞, -)	(∞, -)
1	B,C,D,E,Z	(0,-)*	(1,A)	(2,A)	(∞, -)	(∞, -)	(∞, -)
2	C,D,E,Z	-	(1,A)*	(2,A)	(6,B)	(∞, -)	(∞, -)
3	D,E,Z	-	-	(2,A)*	(6,B)	(12,C)	(∞, -)
4	E,Z	-	-	-	(6,B)*	(8,D)	(12,D)
5	Z	-	-	-	-	(8,D)*	(11,E)
6	№	-	-	-	-	-	(11,E)*

34

Bài tập 2

- Tìm đường đi ngắn nhất giữa A đến các đỉnh trong đồ thị G dưới đây

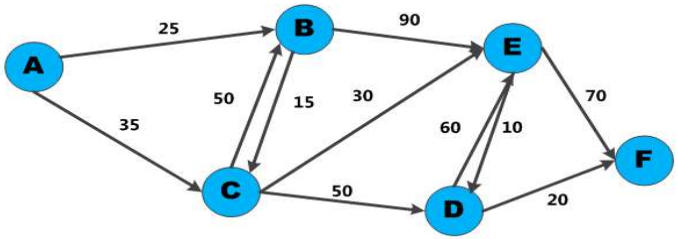


36

36

Bài tập 3

Use Dijkstra's algorithm to find shortest paths from vertex C to others



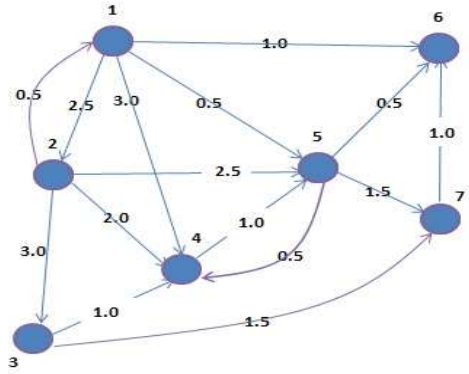
What is "current distance of A" after executing of the algorithm end?

39

39

Bài tập 4

Use Dijkstra's algorithm to find the shortest path from vertex 1 to others.

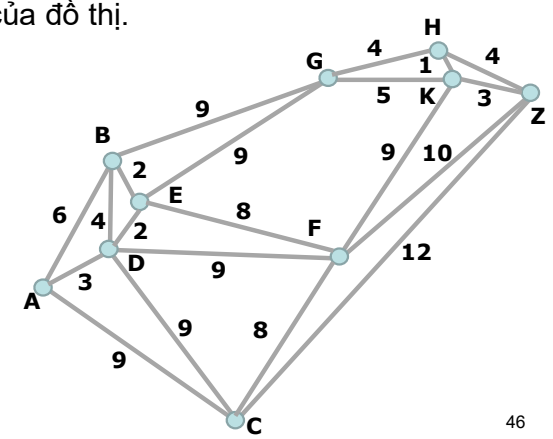


45

45

Bài tập 5

Cho đồ thị có trọng số G.Tìm đường đi ngắn nhất từ A đến mỗi đỉnh của đồ thị.

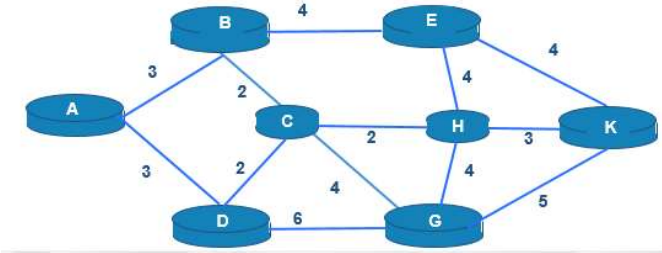


46

46

Bài tập 6

- Cho đồ thị có trọng số G = (V, E), tìm đường đi ngắn nhất giữa A và H.

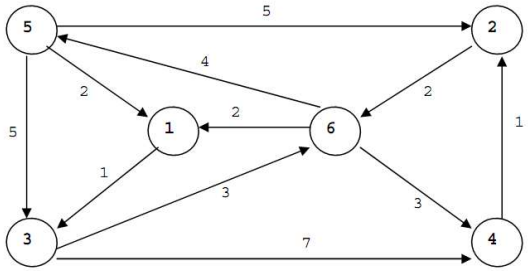


47

47

Bài tập 7

Find the shortest path from the vertex 5 to the vertex 4



48

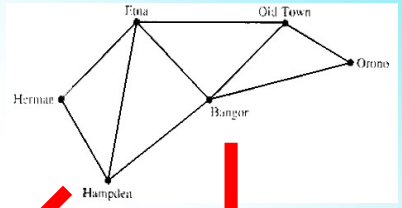
48

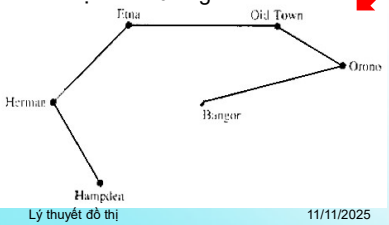
Cây khung (Spanning Tree)

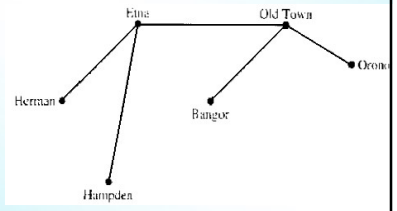
93

Bài toán mở đầu

- Hệ thống đường giao thông ở Maine như hình bên.
- Tuyết đang phủ toàn bộ các con đường.
- Cần khôi phục lại hệ thống giao thông bằng cách cào tuyết một số con đường.
- Không nhất thiết phải cào tuyết hết mọi con đường.



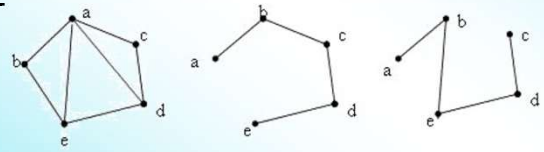




94

Cây khung

- **Định nghĩa:** Cho G là đơn đồ thị. Một cây T được gọi là cây khung của G nếu và chỉ nếu:
  - T là đồ thị con của G
  - T chứa tất cả các đỉnh của G
- **VD:**

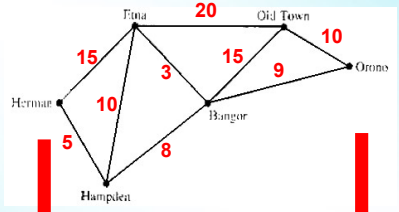


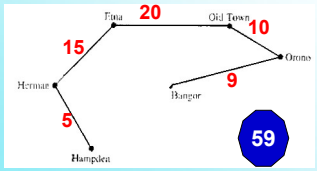
Đồ thị và các cây khung của nó

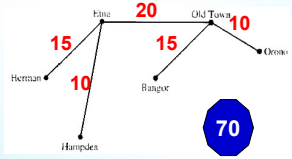
95

Bài toán cây khung nhỏ nhất

- Tìm các con đường để cào tuyết sao cho chi phí là nhỏ nhất





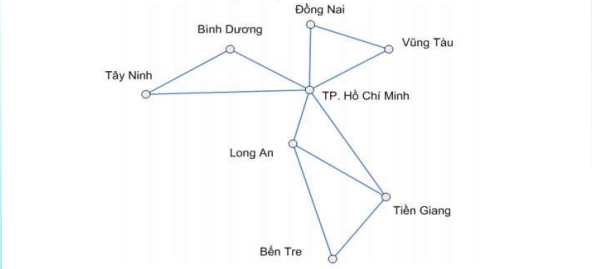


97

Bài toán cây khung nhỏ nhất

MST: *Minimum Span Tree*

- Ứng dụng
  - Bài toán xây dựng hệ thống đường sắt
  - Bài toán nối mạng máy tính



98

Bài toán cây khung nhỏ nhất (tt)

- **Định nghĩa.** Cho đồ thị có trọng số G. Cây khung nhỏ nhất của G (nếu tồn tại) là cây khung có tổng trọng số nhỏ nhất trong số các cây khung của G.
- **Các thuật toán tìm cây khung nhỏ nhất:**
  - Thuật toán Kruskal
  - Thuật toán Prim

Thuật toán Kruskal

- **Ý tưởng:**
  - Lần lượt xét các cạnh theo thứ tự trọng số tăng dần
  - Ứng với mỗi cạnh đang xét, ta thử đưa nó vào cây khung T:
    - Nếu không tạo thành chu trình với các cạnh đã chọn thì chấp nhận cạnh mới này và đưa vào cây.
    - Nếu tạo thành chu trình với các cạnh đã chọn thì bỏ qua và xét cạnh kế tiếp.
  - Cứ tiếp tục như vậy cho đến khi tìm đủ n-1 cạnh để đưa vào cây T

Thuật toán Kruskal (tt)



Trọng số	Cạnh	
4	(3,5)	← Chọn
8	(4,6)	← Chọn
9	(4,5)	← Chọn
14	(5,6)	← Không chọn vì tạo chu trình: 4 5 6 4
16	(3,4)	← Không chọn vì tạo chu trình: 3 4 5 3
17	(1,3)	← Chọn
18	(2,3)	← Chọn. Dừng vì đã đủ cạnh.
20	(2,4)	
33	(1,2)	

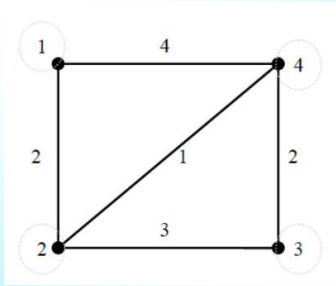


Trọng số	Cạnh	
5	(3,5)	← Chọn
7	(4,6)	← Chọn
9	(4,5)	← Chọn
12	(5,6)	← Không chọn vì tạo chu trình: 4 5 6 4
16	(3,4)	← Không chọn vì tạo chu trình: 3 4 5 3
18	(1,3)	← Chọn
19	(2,3)	← Chọn. Dừng vì đã đủ cạnh.
20	(2,4)	
33	(1,2)	

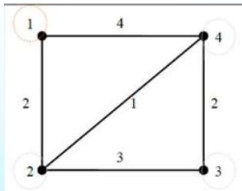
Thuật toán Prim

- Ý tưởng:
  - Xuất phát từ 1 đỉnh bất kỳ. Đưa đỉnh này vào cây khung T.
  - Tại mỗi bước, luôn chọn cạnh có trọng số nhỏ nhất trong số các cạnh liên thuộc với một đỉnh trong T (đỉnh còn lại nằm ngoài T)
  - Đưa cạnh mới chọn và đỉnh đầu của nó vào cây T
  - Lặp lại quá trình trên cho đến khi đưa đủ n-1 cạnh vào T

Exercise(i) Prim



Solution(i)-1

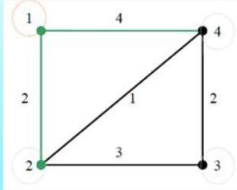


Bước 1: *Chọn tùy ý v ∈ X và khởi tạo Y:= {v}; T := Ø*

Tập X là tập các đỉnh của đồ thị nên

X = {1,2,3,4 }

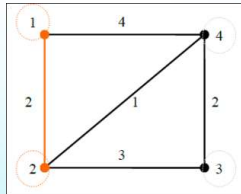
Ta chọn đỉnh 1 làm đỉnh xét đến đầu tiên: Y = {1}; T = Ø. Như vậy X\Y = {2,3,4 }



Bước 2: *Trong số những cạnh e nối đỉnh w với đỉnh v trong Y với w ∈ X\Y và v ∈ Y ta chọn cạnh có trọng lượng nhỏ nhất.*

Cạnh (4,1) và (2,1) nối đỉnh 2 và 4 đến đỉnh 1 trong Y, ta chọn cạnh (2,1) vì nó có trọng nhỏ nhất trong hai cạnh (giá trị là 2).

Solution(i)-2



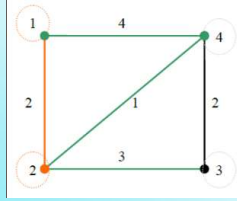
Bước 3: *Gán Y:= Y∪ {w} và T:= T∪ {e}*

Y = Y ∪ {2}

T = T ∪ {(2,1)}

Bước 4: *Nếu T đủ n-1 phần tử thì dừng, ngược lại làm tiếp tục bước 2.*

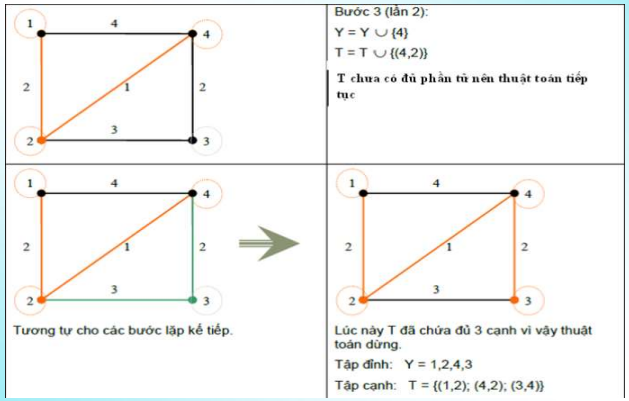
T chỉ có 1 phần tử < n – 1 = 4 – 1 = 3 nên thuật toán chưa dừng.



Bước 2 (lần 2):

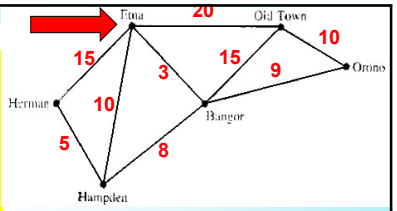
Cạnh (4,1); (4,2); (3,2) là các cạnh nối đỉnh 4; 3 (tập những đỉnh chưa có trong cây) đến 1;2 (tập các đỉnh đã có trong cây). Tương tự, ta chọn cạnh có trọng lượng nhỏ nhất: (4,2).

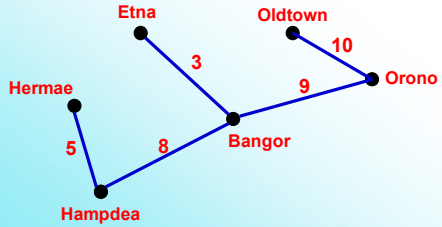
Solution(i)-3



Thuật toán Prim (tt)

- Để biểu diễn lời giải, ta sẽ sử dụng 2 mảng:
  - Mảng d: d[v] dùng để lưu độ dài cạnh ngắn nhất nối với v trong số các cạnh chưa xét.
  - Mảng near: near[v] dùng để lưu đỉnh còn lại (ngoài v) của cạnh ngắn nhất nói ở trên.





v	d[v]	near[v]